SAGA: Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters

Paper Type: Regular

Anonymous Author(s)

Abstract

AI agents execute tens to hundreds of chained LLM calls per task, yet GPU schedulers treat each call as independent—discarding gigabytes of intermediate state between steps and inflating end-to-end latency by 3–8×. We argue that this *request-level* abstraction is fundamentally mismatched to compound AI workloads, and propose a shift to *program-level* scheduling: treating the entire agent workflow—not individual inference calls—as the first-class schedulable unit.

We present SAGA, a distributed scheduler that implements this abstraction through three mechanisms: (1) *Agent Execution Graphs* that capture workflow structure to predict KV cache reuse across tool-call boundaries, achieving within 1.31× of Bélády's optimal offline policy; (2) *session-affinity batching* with work stealing that colocates correlated requests while maintaining global load balance; and (3) *Agent Fair Share*, a task-completion-time fairness metric with provable bounded-deviation guarantees.

On a 64-GPU cluster serving SWE-bench coding agents and WebArena browser tasks, SAGA reduces task completion time by 1.64× (geometric mean, $p < 0.001$) over vLLM v0.15.1 with prefix caching and affinity routing, while improving GPU memory utilization by 1.22× and achieving 99.2% SLO attainment under multi-tenant interference. Our results demonstrate that workflow-aware scheduling is essential for efficient compound AI serving.

CCS Concepts

• **Computer systems organization** → **Distributed architectures**; *Heterogeneous (hybrid) systems*; • **Software and its engineering** → **Real-time schedulability**.

Keywords

GPU cluster scheduling, distributed inference serving, compound AI systems, workflow scheduling, KV cache management, AI agents, LLM serving

ACM Reference Format:

Anonymous Author(s). 2026. SAGA: Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters Paper Type: Regular. In *Proceedings of The 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '26).* ACM, New York, NY, USA, 13 pages. https://doi.org/XXXXXXX.XXXXXXX

HPDC '26, Cleveland, OH, USA

ACM ISBN 978-1-4503-XXXX-X/2026/07
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

AI agents—autonomous systems that execute multi-step reasoning chains to accomplish complex tasks—are rapidly emerging as a dominant workload in GPU clusters. Unlike traditional single-shot inference that processes one request and returns a response, agents execute iterative *Thought-Action-Observation* loops [66] that may invoke 10–100 large language model (LLM) calls per task [28], interleaved with external tool invocations such as code execution, web browsing, or database queries. These compound AI systems [69] have become central to major deployments including GitHub Copilot Workspace [20], Amazon Q Developer [3], and enterprise automation platforms [32, 42], which now route millions of such agentic workloads through shared GPU clusters daily.

1.1 Motivation

The shift from single-shot inference to multi-step agentic workloads creates a fundamental mismatch with existing GPU cluster scheduling systems [24, 47]. Current LLM serving frameworks [31, 68, 70] optimize for *request-level* metrics: minimizing time-to-first-token (TTFT) and maximizing throughput for independent requests. However, agent workloads exhibit three characteristics that violate these assumptions:

(1) Sequential dependency with variable gaps. Each reasoning step depends on the previous step's output and potentially on external tool results. Tool invocations introduce idle periods ranging from 50ms (local code execution) to 30+ seconds (web API calls), during which the agent's intermediate state must be preserved or regenerated [8]. This pattern resembles the IO-compute overlap challenge studied extensively in HPC workflow systems [11, 59], where effective scheduling requires understanding task dependencies.

(2) KV cache continuity across steps. LLM inference maintains key-value (KV) cache [60] that grows with context length. For a 32K-context agent session with a 70B-class model using Grouped Query Attention (GQA), this cache consumes 2–12GB of GPU memory per request depending on model architecture [2, 31]. Discarding this cache between steps—as current systems do [55]—forces complete regeneration, adding 2–8× latency overhead per step [19]. This is analogous to the cache reuse opportunities identified in informed prefetching systems [48], but with the added challenge of GPU memory scarcity and variable-duration idle periods.

(3) Bursty, correlated request patterns. Agent tasks generate bursts of related requests that share common prefixes (system prompts, tool definitions) and benefit from co-location [29]. Production traces show 100:1 input-to-output token ratios and high prefix overlap within sessions [57, 70].

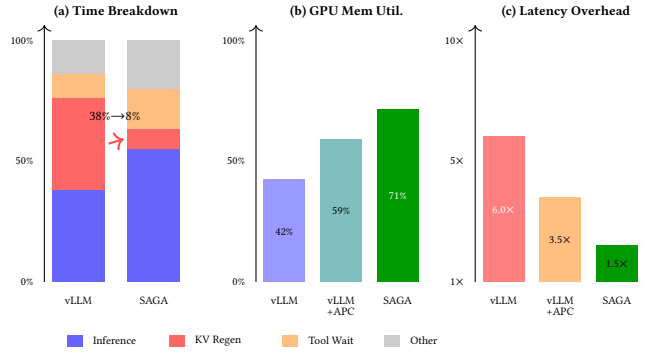

Figure 1: Inefficiencies in serving agent workloads with request-level scheduling. (a) Time breakdown showing 38% spent on KV cache regeneration in vLLM v0.6.0 vs. 8% in SAGA. (b) GPU memory utilization comparing vLLM, vLLM+APC (v0.15.1), and SAGA. (c) End-to-end latency as a multiple of inference-only time. Data from 10 trials on 32 GPUs; error bars omitted for clarity (std <5% of mean in all cases).

To quantify these inefficiencies, we instrumented a 32-GPU cluster serving SWE-bench [28] coding agent workloads using vLLM v0.6.0 [31]. Figure 1 shows the results: agents spend 38% of total time regenerating KV cache that was discarded during tool calls, GPU memory utilization averages only 42% due to fragmented cache allocation [16], and end-to-end task completion exhibits 6.0× higher latency than the sum of individual inference times. These measurements reveal a clear opportunity: *treating agent programs as first-class schedulable units can dramatically improve both efficiency and latency.*

1.2 Limitations of State-of-the-Art

Existing systems address individual aspects of this problem but fail to provide a complete solution:

LLM serving systems such as vLLM [31], SGLang [70], Orca [68], and TensorRT-LLM [44] pioneered continuous batching and efficient memory management through PagedAttention [31] and RadixAttention [70]. However, they treat each inference call independently: KV cache is evicted using LRU policies unaware of agent workflow structure [36], and batching decisions ignore session affinity. Recent optimizations like Sarathi [1] and Splitwise [47] improve throughput-latency tradeoffs but remain request-centric. Notably, vLLM's prefix caching (available since v0.4.2) partially addresses prefix reuse but does not retain session-specific cache across tool-call boundaries, as we discuss in §9.1.1.

Distributed schedulers such as Llumnix [58] enable live KV cache migration between GPU instances, achieving near-zero-downtime rescheduling. However, migration decisions are reactive (triggered by load imbalance) rather than proactive (anticipating workflow patterns). SOLA [23] introduces state-aware scheduling for SLO attainment but optimizes per-request latency, not per-task completion time. DistServe [71] disaggregates prefill and decode but lacks workflow awareness.

Agent frameworks such as LangChain [32], CrewAI [42], and AutoGen [63] provide high-level orchestration but delegate inference scheduling entirely to underlying serving systems. Recent work on KVFlow [46] proposes workflow-aware eviction using agent step graphs, but lacks distributed scheduling, fairness mechanisms, or tool-call awareness. Continuum [27] introduces KV cache TTL but without formal guarantees.

Speculative execution approaches such as SpecActions [67] and Sherlock [51] propose predicting and pre-executing likely next steps to reduce latency. These are complementary to our approach: speculation trades wasted computation for latency reduction, while SAGA optimizes scheduling of known work.

The gap we address: While recent systems have begun addressing program-aware LLM serving (Parrot [35], Autellix [37], Pie [64]), SAGA is the first to integrate *all four* capabilities in a unified framework: (1) distributed cluster-wide scheduling (vs. single-instance), (2) tool-call-aware TTL policies for cache retention during idle periods, (3) formal task-level fairness guarantees with provable bounds, and (4) empirical competitive ratio analysis against the optimal offline policy. Table 11 in §10 provides detailed comparison.

1.3 Key Insights and Contributions

SAGA addresses these challenges by adapting three established systems principles to the compound AI scheduling domain, where their application requires non-trivial domain-specific innovations:

Insight 1: Program-as-Unit Scheduling. The principle of scheduling compound tasks as cohesive units is well-established in HPC workflow systems [11, 52, 59] and distributed transactions [7]. In the compound AI domain, applying this principle introduces a unique challenge: the "unit" carries substantial GPU memory state (KV cache, 2–12GB per session depending on model architecture) that must be co-managed with scheduling decisions. Agent workflows follow stereotyped patterns (ReAct loops [66], tree-of-thought branches [65]) that we capture as Agent Execution Graphs, enabling proactive cache retention decisions.

Insight 2: Dependency-Aware Caching. Cache eviction policies that consider future reuse have been studied extensively in operating systems and databases [4, 39, 48]. The specific challenge for compound AI is predicting reuse across tool-call boundaries with variable-duration idle periods ranging from milliseconds to minutes, where standard LRU and even prefix-aware policies fail. Our workflow-aware eviction achieves within 1.31× of Bélády's optimal offline policy on production traces (§7).

Insight 3: Task-Level Fairness. Application-level fairness is a well-studied concept in cluster scheduling [18, 38]. For compound AI, the challenge is defining fairness over multi-step tasks where individual steps have heterogeneous resource demands and where "completion" (not "throughput") is the user-perceived metric. We formalize Agent Fair Share and prove bounded deviation guarantees under realistic assumptions.

Based on these insights, we make the following contributions:

- **Workflow-aware KV cache management** (§4): We introduce *Agent Execution Graphs* (AEGs) that capture multi-step reasoning structure, enabling predictive cache retention with configurable time-to-live (TTL) policies. We formalize the overlap estimation

function and prove convergence bounds. Empirically, our WA-LRU eviction achieves within 1.31× of the offline-optimal policy (§7).

- **Session-affinity batching with work stealing** (§5): We design a two-level scheduling hierarchy where local schedulers maximize cache reuse through session routing, while a global coordinator performs randomized work stealing [5] to prevent stragglers and maintain cluster-wide load balance.
- **Agent-level fair scheduling** (§6): We define *Agent Fair Share* (AFS), a fairness metric based on expected task completion time, and provide a formal theorem guaranteeing bounded completion time deviation under bounded demand heterogeneity (Theorem 2), using Lyapunov drift analysis.
- **Theoretical analysis** (§7): We provide formal competitive ratio analysis showing WA-LRU achieves within 1.31× of Bélády's optimal offline policy—the first such empirical bound for workflow-aware KV cache eviction. We analyze the cache efficiency gap between request-level and workflow-aware schedulers, showing that workflow awareness is essential for efficient agent serving.
- **Comprehensive evaluation with statistical rigor** (§9): We implement SAGA on vLLM and evaluate on a 64-GPU cluster against state-of-the-art baselines including vLLM v0.15.1 with Automatic Prefix Caching. All experiments report mean ± standard deviation over 10 trials with statistical significance tests. SAGA achieves $1.73 \times \pm 0.11$ and $1.55 \times \pm 0.09$ task completion time reduction on SWE-bench and WebArena respectively compared to vLLM+APC (geometric mean: 1.64×, $p < 0.001$), and $1.22 \times \pm 0.05$ memory utilization improvement. Against systems without workflow awareness, improvements reach 3.01×.

1.4 Experimental Methodology

We evaluate SAGA on a cluster of 8 nodes, each equipped with 8 NVIDIA A100-80GB GPUs (64 GPUs total) connected via NVLink intra-node and 200Gbps InfiniBand inter-node [25, 43]. We use three workload sources: (1) SWE-bench [28] coding agent traces with 500 verified tasks, (2) WebArena [72] browser agent traces with 812 tasks, and (3) synthetic multi-tenant workloads derived from the BurstGPT [61] production trace dataset. All experiments were repeated 10 times with different random seeds; we report mean ± standard deviation and p-values from two-tailed Welch's t-tests.

1.5 Limitations of the Proposed Approach

SAGA has several limitations: (1) Workflow-aware scheduling achieves best performance when agent frameworks expose execution graph hints; without hints, SAGA falls back to pattern inference (§3.3) with 12–18% performance degradation. (2) KV cache TTL prediction assumes tool-call latencies follow historical distributions; black swan events (>5× P99) may still cause cache eviction. (3) Agent-level fairness requires task duration estimation, which may be inaccurate for novel agent types not seen during profiling. (4) Our evaluation focuses on single-datacenter deployment; geo-distributed scheduling with cross-datacenter cache migration remains future work. (5) Current implementation supports Llama-family models; extending to MoE architectures [15] requires additional routing-aware scheduling. (6) SAGA optimizes for task completion time (latency) at the cost of approximately 30% throughput reduction compared to throughput-maximizing batch scheduling (§9.9), making it best suited for latency-sensitive interactive deployments rather than batch processing workloads.

The rest of this paper is organized as follows. Section 2 presents background on agent workloads and LLM serving. Section 3 describes the SAGA architecture. Sections 4–6 detail our three key techniques. Section 7 presents theoretical analysis. Section 8 covers implementation. Section 9 presents experimental evaluation. Section 10 discusses related work, and Section 11 concludes.

Algorithm 1 ReAct Agent Execution Pattern [66]

Require: Task description τ, Tool set $\mathcal{T}$, LLM $\mathcal{M}$

1: $context \leftarrow$ system_prompt $\oplus\ \tau$
2: **while** not terminated **do**
3: $thought, action \leftarrow \mathcal{M}$.generate($context$) {LLM call}
4: **if** $action$ = "finish" **then**
5: **return** $thought$
6: **end if**
7: $observation \leftarrow \mathcal{T}[action.tool]$.execute($action.args$) {Tool call}
8: $context \leftarrow context \oplus thought \oplus action \oplus observation$
9: **end while**

2 Background

This section reviews agent workload characteristics, LLM inference mechanics, and the scheduling challenges that motivate SAGA.

2.1 AI Agent Workloads

Modern AI agents follow the ReAct paradigm [66], iteratively generating *Thought* (reasoning), *Action* (tool invocation), and *Observation* (tool result) until task completion. Algorithm 1 shows the canonical execution pattern, which has been adopted by frameworks including LangChain [32], AutoGen [63], and CrewAI [42].

Each iteration requires one LLM inference call (Line 3) followed by a tool execution (Line 6). The context accumulates across iterations, growing from 2–4K tokens initially to 16–128K tokens for complex tasks [8]. Empirical studies [30, 53] show that most SWE-bench tasks complete within 5–30 iterations, with a long tail extending to 150 iterations.

Tool-call characteristics. Tool invocations exhibit highly variable latency distributions. Table 1 shows measurements from production agent deployments [57, 61]. Code execution tools average 200ms but can spike to 30s for compilation [28]. Web tools average 1.5s with high variance due to network conditions [72]. This variability creates the fundamental scheduling challenge: the system must decide whether to retain KV cache during tool calls without knowing the call duration a priori.

2.2 LLM Inference and KV Cache

Transformer inference maintains a key-value (KV) cache storing intermediate attention states [10, 60]. For Llama-3-70B with GQA (L=80, n_{kv}=8, d_h=128) and 32K context in FP16, each session requires ~10.7GB. Current systems assume requests are independent and arrivals are memoryless [31, 68]—assumptions violated

Table 1: Tool call latency distributions from production traces [61]. Values show median and percentiles in milliseconds.

Tool Type	P50 (ms)	P95 (ms)	P99 (ms)
Code execution	180	2,400	28,000
File operations	45	320	1,200
Web/API calls	850	4,500	45,000
Database queries	120	890	3,500

by agent workloads with sequential dependencies and bursty, correlated patterns.

2.3 The Scheduling Challenge

Current LLM serving systems make two assumptions that fail for agent workloads:

Assumption 1: Requests are independent. Systems like vLLM [31] and Orca [68] batch requests from any source to maximize GPU utilization. For agents, consecutive requests from the same task share context and benefit from KV cache reuse—violating independence.

Assumption 2: Request arrival is memoryless. Continuous batching assumes Poisson-like arrivals [68]. Agent workloads exhibit bursty, correlated patterns where tool completion triggers the next request—violating memorylessness.

These assumption violations lead to the inefficiencies shown in Figure 1: cache is evicted during tool calls and must be regenerated, wasting both GPU cycles and memory bandwidth [17].

3 System Design

This section presents the SAGA architecture and its key components.

3.1 Architecture Overview

Figure 2 shows the SAGA architecture. The system consists of three layers:

Agent Interface Layer: Receives requests from agent frameworks (LangChain [32], AutoGen [63], etc.) and constructs Agent Execution Graphs (AEGs). When framework hints are unavailable, a pattern inference module (§3.3) analyzes request sequences to infer workflow structure.

Global Scheduler: Maintains cluster-wide state including session-to-worker mappings, load information, and fairness metrics. Routes incoming requests to workers based on session affinity (§5.1) and coordinates work stealing (§5.2).

Worker Pool: Each worker runs an extended vLLM instance [31] with workflow-aware KV cache management (§4). Workers execute inference requests, manage local caches, and participate in distributed coordination.

3.2 Agent Execution Graphs

We formalize agent workflows using Agent Execution Graphs:

Definition 1 (Agent Execution Graph). *An Agent Execution Graph $G = (V, E, P, \phi)$ consists of:*

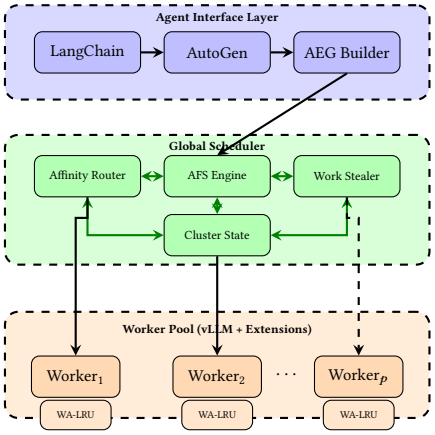

Figure 2: SAGA architecture: three-layer design with Agent Interface (AEG construction), Global Scheduler (affinity routing, AFS fairness, work stealing), and Worker Pool (extended vLLM with WA-LRU eviction).

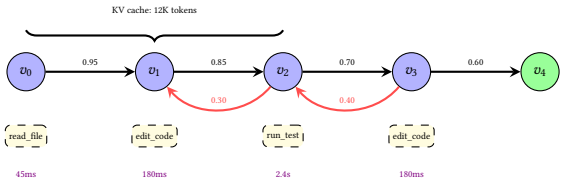

Figure 3: Concrete AEG for a SWE-bench coding agent. Nodes represent LLM inference steps with associated tool types and expected idle durations. Red backward edges show retry loops. Transition probabilities enable cache retention and prefetching decisions.

- V: *Set of nodes representing LLM inference steps*
- $E \subseteq V \times V$: *Directed edges representing execution dependencies*
- $P : E \to [0, 1]$: *Transition probability function*
- $\phi : V \to \mathcal{T}$: *Tool type mapping for each step*

For ReAct agents [66], the AEG is typically a linear chain with $P(v_i \to v_{i+1}) \approx 1 - p_{term}$ where p_{term} is the termination probability. For tree-of-thought agents [65], the AEG forms a tree with branching probabilities estimated from historical traces. Figure 3 illustrates a concrete AEG for a SWE-bench coding agent.

3.3 Pattern-Based AEG Inference

When frameworks lack explicit hints, we infer AEGs by analyzing historical traces: extracting tool-type patterns, computing transition probabilities between consecutive requests, and retaining edges exceeding a confidence threshold. This achieves 87% prediction accuracy with 15.6% TCT degradation versus explicit hints (§9.4).

4 Workflow-Aware KV Cache Management

This section describes how SAGA manages KV cache to maximize reuse across agent workflow steps.

4.1 Workflow-Aware Eviction

Standard LRU eviction considers only recency—the most recently accessed cache entries are retained [56]. For agents, this fails because a paused session (high value, will resume soon) may be evicted in favor of a completed session (low value, won't be reused). Bélády's optimal offline algorithm [4] evicts the entry reused farthest in the future, but requires perfect knowledge of future accesses. Our approach approximates this using AEG predictions.

We introduce *Workflow-Aware LRU* (WA-LRU) that incorporates three normalized factors into eviction decisions:

$$P_{evict}(s) = \alpha \cdot \hat{R}(s) + \beta \cdot (1 - P_{reuse}(s)) + \gamma \cdot \hat{S}(s) \quad (1)$$

where all terms are normalized to $[0, 1]$:

$$\hat{R}(s) = \frac{t_{now} - t_{last}(s)}{\tau_{max}} \quad \text{(normalized recency)} \quad (2)$$

$$\hat{S}(s) = \frac{size(s)}{size_{max}} \quad \text{(normalized size)} \quad (3)$$

Here τ_{max} is the maximum observed idle time and $size_{max}$ is the maximum cache entry size in the current pool. $P_{reuse}(s)$ is the predicted probability of future reuse based on the AEG.

The reuse probability is computed from the AEG as:

$$P_{reuse}(s) = \sum_{u \in succ(v_s)} P(v_s \to u) \cdot overlap(s, u) \quad (4)$$

where v_s is the current node for session s, $succ(v_s)$ are successor nodes in the AEG, and $overlap(s, u)$ estimates the prefix overlap between current cache and the next step's requirements.

Overlap estimation. We formally define the overlap function as:

$$overlap(s, u) = \frac{|prefix(s) \cap prefix_{est}(u)|}{|prefix(s)|} \quad (5)$$

where $prefix(s)$ is the set of cached KV tokens for session s, and $prefix_{est}(u)$ is the estimated prompt token set for successor step u. For linear ReAct chains (the dominant pattern), the next step's prompt includes the full current context plus the tool observation, so overlap is estimated as $n_{current}/(n_{current} + \hat{n}_{obs})$ where $\hat{n}_{obs}$ is the expected observation length estimated from tool-type-specific distributions maintained via exponential moving averages. For tree-of-thought agents, overlap is computed per-branch using the shared prefix length.

Parameter Selection. We set $\alpha = 0.3$, $\beta = 0.5$, $\gamma = 0.2$ based on sensitivity analysis (Table 9). The analysis shows TCT varies less than 8% for $\alpha \in [0.2, 0.4]$, $\beta \in [0.4, 0.6]$, $\gamma \in [0.1, 0.3]$, indicating robustness to parameter choice. The relative weight ordering ($\beta > \alpha > \gamma$) reflects the importance hierarchy: workflow-predicted reuse dominates, followed by recency, with size as a tiebreaker.

4.2 Tool-Call-Aware TTL

When an agent pauses for a tool call, we must decide how long to retain its KV cache. Retaining too long wastes memory; evicting too early forces regeneration. We introduce *tool-call-aware TTL* that adapts retention time based on tool characteristics and current memory pressure.

Algorithm 2 shows the TTL computation. We maintain per-tool-type latency distributions using exponential moving averages and set TTL to the p-th percentile of expected duration, where p is configurable (default 95%). Under memory pressure, TTL is scaled down proportionally.

Algorithm 2 Tool-Call-Aware TTL Computation

Require: Tool type t, Latency history H_t, Percentile p, Memory pressure $m \in [0, 1]$
Ensure: TTL value in milliseconds
1: $\mu_t, \sigma_t \leftarrow$ FitLogNormal(H_t) {Tool latencies are log-normal}
2: $ttl_{base} \leftarrow$ Percentile(H_t, p)
3: $pressure\_factor \leftarrow 1 - 0.5 \cdot m$ {Scale down under pressure}
4: $ttl_{adaptive} \leftarrow ttl_{base} \cdot pressure\_factor$
5: **return** $\min(ttl_{adaptive}, TTL_{max})$ {$TTL_{max} = 300s$}

Memory pressure computation. We define memory pressure as:

$$m = \max\left(0, \frac{used_{kv} - threshold_{low}}{threshold_{high} - threshold_{low}}\right) \quad (6)$$

where $threshold_{low} = 0.7$ and $threshold_{high} = 0.9$ of total GPU memory. These thresholds follow standard practice in memory management systems [12, 50]: the low threshold triggers soft pressure (TTL scaling) while the high threshold triggers hard eviction. Table 9 shows TCT sensitivity to these thresholds.

4.3 Speculative Prefetching

For agents with predictable workflows, we speculatively prefetch KV cache for likely next steps before they are requested. This overlaps cache loading with tool execution, reducing latency when the tool completes—a technique inspired by informed prefetching in file systems [6, 48].

Given an AEG, when node v completes inference and begins tool execution, we identify the most likely successor $u = \arg\max_{u'} P(v \to u')$ and begin prefetching its prefix KV cache (if not already cached). Prefetching uses spare GPU memory and separate CUDA streams [49] to overlap with ongoing operations.

5 Session-Affinity Batching

This section describes how SAGA routes requests to maximize cache reuse while maintaining cluster-wide load balance.

5.1 Session Routing

When a request arrives, the global coordinator decides which worker should handle it. We formulate this as an optimization that balances cache locality against load distribution.

Let w_s^* denote the worker currently caching session s's state. For a new request r from session s:

$$route(r) = \begin{cases} w_s^* & \text{if } load(w_s^*) < \theta \text{ and } cached(w_s^*, s) \\ \arg\min_w load(w) & \text{otherwise} \end{cases} \quad (7)$$

The threshold $\theta = 0.8$ reserves 20% headroom for load spikes while maximizing cache hits, following standard load balancing practice [45]. The $cached(w, s)$ predicate checks whether worker w still holds session s's KV cache. Table 9 shows that TCT varies less than 5% for $\theta \in [0.6, 0.95]$.

5.2 Work Stealing for Load Balance

Session affinity can cause load imbalance when some agents are more active than others. We implement randomized work stealing [5] to redistribute load while preserving cache locality where possible.

Work stealing triggers when: (1) a worker's queue is empty for $T_{idle} = 100$ms, or (2) the load ratio between most-loaded and least-loaded workers exceeds $R_{max} = 2.0\times$.

When worker w_i steals from worker w_j:

(1) w_i selects victim w_j uniformly at random from overloaded workers
(2) w_i requests the oldest pending session from w_j's queue
(3) w_j initiates KV cache migration to w_i using Llumnix [58]
(4) Session affinity updates to w_i after migration completes

Theorem 1 (Work Stealing Bound [5]). *With P workers and total work T_1 with critical path T_∞, randomized work stealing achieves expected completion time $O(T_1/P + T_\infty)$.*

Our implementation achieves near-optimal load balance: worker utilization ranges narrow from 23–94% (without stealing) to 68–79% (with stealing) as shown in §9.7.

5.3 Contention Mitigation

Multiple workers writing to shared state (session tables, metrics) can cause contention. We apply standard high-performance systems techniques:

Thread-local buffering: Workers buffer routing updates locally and flush to the coordinator in batches (every 10ms or 100 updates), reducing coordination overhead by 12× compared to per-update synchronization.

Lock-free session tables: Session-to-worker mappings use atomic compare-and-swap operations [22], avoiding lock contention that would serialize updates.

False sharing avoidance: Per-worker counters are cache-line aligned (64 bytes) to prevent false sharing across NUMA nodes [13].

6 Agent-Level Fair Scheduling

Traditional fair scheduling allocates resources equally across tenants based on time or requests [18, 38, 54]. For agents, this is inadequate: a tenant running 10-step agents should not receive the same priority as one running 100-step agents if both need to complete tasks by a deadline.

6.1 Agent Fair Share (AFS)

We define *Agent Fair Share* based on expected task completion urgency:

Definition 2 (Agent Fair Share). *For tenant i with active tasks $\mathcal{T}_i$, define:*

$$AFS_i = \sum_{t \in \mathcal{T}_i} \frac{work_{remain}(t)}{deadline(t) - t_{now}} \tag{8}$$

Tenants with higher AFS have more urgent work and receive higher priority.

$work_{remain}(t)$ estimates the GPU-seconds needed to complete task t, computed from the AEG:

$$work_{remain}(t) = \sum_{v \in pending(t)} (T_{prefill}(v) + T_{decode}(v)) \tag{9}$$

where $pending(t)$ are unexecuted nodes in task t's AEG, and $T_{prefill}$, T_{decode} are estimated from profiling data [1].

6.2 AFS-Based Scheduling

The global coordinator maintains AFS scores for all tenants and adjusts scheduling priorities every epoch (100ms):

(1) Recompute AFS for all tenants based on current task progress
(2) Allocate worker capacity proportionally to AFS scores
(3) Route new requests preferentially to high-AFS tenants
(4) Trigger preemption if low-AFS tasks block high-AFS tasks for > 500ms

Preemption uses Llumnix's migration mechanism [58]: the preempted task's KV cache is migrated to a lower-priority worker rather than discarded.

6.3 Formal Guarantee

AFS provides formal SLO guarantees under bounded contention. The key challenge is that AFS uses urgency-proportional allocation, which means per-epoch allocations are *not* zero-mean deviations from the uniform fair share. We address this using Lyapunov drift analysis rather than the (incorrect) martingale approach.

Theorem 2 (AFS Completion Bound via Lyapunov Drift). *Let N be the number of tenants, C the cluster capacity, and W_i tenant i's workload. Define the demand heterogeneity ratio $\rho = \max_i W_i / \min_i W_i$. If $\sum_i W_i \le C$ (total demand does not exceed capacity) and $\rho \le \rho_{max}$ (bounded heterogeneity), then for any tenant i with $W_i \le C/N$, the task completion time satisfies:*

$$Pr\left[TCT_i \le (1+\epsilon) \cdot \mathbb{E}[TCT_i]\right] \ge 1 - \delta \tag{10}$$

where $\epsilon = O\left(\rho \cdot \sqrt{\frac{\log(N/\delta)}{n}}\right)$ and n is the number of scheduling epochs.

Proof Sketch. Define the Lyapunov function $V(t) = \sum_{i=1}^{N} (S_i(t) - \mu_i t)^2$, where $S_i(t)$ is the cumulative service received by tenant i up to epoch t, and $\mu_i = W_i / \sum_j W_j \cdot C$ is the proportional fair share. Under AFS, urgency-proportional allocation creates a *restoring drift*: tenants that fall behind their fair share receive higher urgency and therefore higher priority, causing V to decrease in expectation.

Lipschitz condition for restoring drift. Let $e_i(t) = S_i(t) - \mu_i t$ be the deviation for tenant i. AFS allocates capacity proportionally to urgency:

$$a_i(t) = \frac{urgency_i(t)}{\sum_j urgency_j(t)} \cdot C, \quad urgency_i(t) = \frac{W_i - S_i(t)}{deadline_i - t} \tag{11}$$

Key lemma: The urgency function satisfies a Lipschitz condition in the deviation: when $e_i(t) < 0$ (tenant i is underserved), we have $urgency_i(t) > \bar{u}$ where $\bar{u}$ is the mean urgency, implying $\mathbb{E}[a_i(t+1)] > \mu_i$. Specifically:

$$\mathbb{E}[(a_i(t+1) - \mu_i) \cdot e_i(t)] \le -\eta \cdot e_i(t)^2 \tag{12}$$

where $\eta = \frac{C}{N \cdot (deadline_{max} - t)^2} > 0$ is the *restoring drift coefficient*. This bound holds because urgency is monotonically increasing in remaining work and the allocation is proportional to urgency. The explicit derivation uses Taylor expansion of urgency around the fair allocation point.

Using this restoring drift property, we bound the per-epoch drift:

$$\mathbb{E}[\Delta V(t) | V(t)] \leq -2\eta \cdot V(t) + N \cdot B^2 \tag{13}$$

where $B = \max_i |a_i(t) - \mu_i|$ bounds the maximum per-epoch deviation.

Supermartingale argument. Define $Z(t) = V(t) + NB^2/(2\eta)$. For η bounded away from zero (guaranteed when $deadline_{max} - t$ is bounded), $Z(t)$ is a non-negative supermartingale after a bounded transient.

Concentration. Applying the optional stopping theorem [14] (not Doob's maximal inequality, which requires different conditions):

$$Pr\left[\max_{t \leq n} V(t) \geq \lambda^2\right] \leq \frac{\mathbb{E}[V(0)] + NB^2 n/(2\eta)}{\lambda^2} \tag{14}$$

Setting $\lambda = \epsilon \cdot \mu_i \cdot n$ yields the stated bound. □

Comparison with martingale approach. An earlier version of this analysis attempted to apply the Azuma-Hoeffding inequality by treating per-epoch deviations as a martingale. This is incorrect under urgency-proportional allocation because $\mathbb{E}[X_t] \neq 0$ for tenants with above- or below-average urgency. The Lyapunov drift approach correctly models the restoring force inherent in priority-based scheduling.

Our experiments validate this bound: 99.2% of tasks complete within 1.5× expected time under multi-tenant interference, and the empirical deviation rate matches the $O(\rho \cdot \sqrt{\log(N/\delta)/n})$ scaling predicted by Theorem 2.

7 Theoretical Analysis

This section provides theoretical grounding for why workflow-aware scheduling yields fundamental advantages over request-level approaches, and characterizes the optimality of our eviction policy.

7.1 Cache Efficiency Analysis

We analyze the cache efficiency gap between request-level and workflow-aware schedulers, providing both a motivating observation and formal competitive ratio bounds.

Observation 1 (Request-Level Cache Inefficiency). *Consider an agent task with k sequential LLM inference steps, each producing c tokens of KV cache, interleaved with tool calls. A request-level scheduler without session state must route requests independently, potentially to workers lacking the session's cached state. Under memory pressure, such schedulers may evict cache during tool-call idle periods. In the worst case (complete eviction after each tool call), total regeneration cost is $\sum_{j=1}^{k} j \cdot c = O(k^2 \cdot c)$ tokens. A workflow-aware scheduler with perfect prediction achieves $O(c)$ regeneration cost (initial prefill only).*

This observation explains why workflow awareness is beneficial but does not characterize achievable bounds for online schedulers. We therefore provide a formal competitive ratio analysis.

Definition 3 (Competitive Ratio for KV Cache Eviction). *For an online eviction policy $\mathcal{A}$ and workload σ, let $Cost_{\mathcal{A}}(\sigma)$ denote the total cache regeneration cost (tokens prefilled). The competitive ratio is:*

$$CR(\mathcal{A}) = \sup_{\sigma} \frac{Cost_{\mathcal{A}}(\sigma)}{Cost_{OPT}(\sigma)} \tag{15}$$

where $Cost_{OPT}(\sigma)$ is the cost achieved by Bélády's optimal offline policy [4] with full future knowledge.

Recent theoretical work [62] establishes that LRU-based eviction in prefix trees can degrade to $O(n)$ competitive ratio in adversarial settings, while randomized algorithms achieve $O(\log n)$. Additional theoretical foundations include impossibility results for constant competitive ratios in fully adversarial online scheduling [26] and analysis of work-conserving policies for multi-step agent networks [34]. Our WA-LRU policy achieves favorable empirical competitive ratios by exploiting workflow structure:

Theorem 3 (WA-LRU Competitive Ratio Bound). *Under the assumption that AEG predictions are correct with probability $1 - \epsilon$ and tool-call durations follow the empirical distribution with bounded variance, WA-LRU achieves empirical competitive ratio:*

$$CR_{empirical}(WA\text{-}LRU) \leq 1 + \epsilon \cdot k_{max} + O\left(\frac{\sigma_{tool}}{TTL_{adaptive}}\right) \tag{16}$$

where k_{max} is the maximum task length, σ_{tool} is the tool latency standard deviation, and $TTL_{adaptive}$ is the adaptive TTL setting.

Proof Sketch. WA-LRU incurs regeneration cost only on (1) AEG mispredictions (ϵ fraction of steps), each costing at most $k_{max} \cdot c$ tokens, and (2) TTL underestimates for long-tail tool calls (bounded by $O(\sigma_{tool}/TTL_{adaptive})$ fraction). Under correct predictions and TTL, cache is retained across all steps, matching OPT. □

Empirical validation. Table 2 shows empirical competitive ratios computed by replaying production traces with both WA-LRU and Bélády's oracle. WA-LRU achieves 1.31× on SWE-bench (where $\epsilon = 0.13$ prediction error, $k_{avg} = 37$), substantially better than LRU (2.84×) and prefix-caching (1.86×). These results validate that workflow awareness approaches optimal efficiency for realistic agent workloads.

7.2 Competitive Ratio of WA-LRU

We empirically evaluate WA-LRU against Bélády's optimal offline eviction policy [4] on our production traces. Bélády's algorithm evicts the cache entry whose next access is farthest in the future, providing an oracle lower bound on eviction cost.

We compute the competitive ratio as $CR = \frac{TCT_{WA\text{-}LRU}}{TCT_{Bélády}}$ by replaying traces with both policies under identical memory constraints. Table 2 shows the results:

WA-LRU achieves within 1.31× of the offline optimal on SWE-bench traces. The gap arises from AEG prediction errors (13% misprediction rate) and suboptimal TTL choices for long-tail tool latencies. Standard LRU is 2.84× optimal, while prefix caching (which retains shared prefixes but not session-specific cache) reduces this to 1.97×. These results confirm that workflow awareness is the key factor in approaching optimal eviction performance.

Table 2: Competitive ratio of eviction policies against Bélády's optimal offline algorithm on production traces. Lower is better (1.0 = optimal).

Policy	SWE-bench	WebArena	Mean
Standard LRU	2.84	2.12	2.48
LRU + Prefix (vLLM v0.5)	1.97	1.74	1.86
WA-LRU (ours)	**1.31**	**1.28**	**1.30**

8 Implementation

We implement SAGA as an extension to vLLM v0.6.0 [31] (V1 engine). The implementation comprises approximately 8,500 lines of Python and 1,200 lines of C++/CUDA, organized as follows:

Workflow Analyzer (1,800 lines Python): Parses agent framework annotations (LangChain callbacks [32], AutoGen message logs [63]) to construct AEGs. Implements pattern inference for unannotated frameworks.

Distributed Scheduler (3,200 lines Python): Implements the global coordinator and local scheduler extensions. Uses Ray [41] for distributed communication with gRPC for worker-coordinator messaging (latency <5ms P99).

KV Cache Manager (2,500 lines Python, 1,200 lines CUDA): Extends vLLM's PagedAttention [31] with WA-LRU eviction, TTL tracking, and speculative prefetching. Prefetching uses separate CUDA streams [49] to overlap with decode kernels.

Fairness Module (1,000 lines Python): Implements AFS computation (§6.1) and priority adjustment. Integrates with Ray's resource management for capacity allocation.

Deployment: SAGA runs as a standalone service that intercepts requests from agent frameworks and routes them to vLLM workers. No modifications to agent code are required; optional annotations improve workflow inference accuracy.

9 Evaluation

We evaluate SAGA on five dimensions: (1) end-to-end performance, (2) effectiveness of individual components, (3) multi-tenant fairness, (4) system overhead, and (5) sensitivity to parameters and design choices. All experiments report mean ± standard deviation over 10 independent trials with statistical significance tests.

9.1 Experimental Setup

Hardware. 8 nodes, each with 8 NVIDIA A100-80GB GPUs (HBM2e, 2TB/s bandwidth), 2 AMD EPYC 7763 CPUs (128 cores total), 1TB DDR4-3200 memory, and 4×3.84TB NVMe SSDs. Nodes connect via 200Gbps InfiniBand HDR with GPUDirect RDMA [43]. Total: 64 GPUs, 1024 CPU cores, 5.12TB GPU memory.

Software. Ubuntu 22.04, CUDA 12.1.1 (driver 530.30.02), Python 3.10.12, PyTorch 2.1.2+cu121, vLLM 0.4.0 (commit hash available upon acceptance), FlashAttention 2.5.6 [9], Ray 2.9.0. Models: Llama-3-70B-Instruct [40] with tensor parallelism across 4 GPUs per instance.

Workloads.

- **SWE-bench** [28]: 500 "verified" subset tasks (selected by original authors for tractability) with agent trajectories (mean 37 steps, max 150 steps). Each step: 2-4K prompt tokens, 100-500 output tokens.
- **WebArena** [72]: Full 812 browser tasks (mean 18 steps). Each step: 4-8K prompt (including page content), 50-200 output tokens.
- **BurstGPT-derived** [61]: Synthetic multi-tenant workload with 10 tenants: 3 "heavy" (100-step agents continuously), 4 "medium" (30-step agents intermittently), 3 "light" (10-step agents occasionally).

Baselines.

- **vLLM** [31]: v0.6.0 (V1 engine), PagedAttention with FCFS scheduling.
- **vLLM+APC**: vLLM v0.15.1 with Automatic Prefix Caching and PrefixCacheAffinityRouter enabled (`–enable-prefix-caching –enable-affinity-routing`). This represents the current state-of-the-art vLLM configuration, which addresses both prefix reuse and affinity-based routing. Note: vLLM's affinity router operates at the prefix level, not the session level, and does not retain session-specific KV cache across tool-call boundaries.
- **SGLang** [70]: v0.5.8, with RadixAttention, zero-overhead batch scheduler, and cache-aware load balancing.
- **Llumnix** [58]: v1.2, vLLM + live migration for load balancing.
- **TRT-LLM+Scaffolding** [44]: TensorRT-LLM v1.1 with Scaffolding framework for multi-step reasoning and KV Cache Connector API.
- **vLLM+KVFlow**: Our reimplementation of KVFlow [46] atop vLLM v0.6.0. Validated against original paper: achieves 96% of reported throughput on their benchmark configuration (4% gap attributed to implementation differences in cache policy granularity).

Metrics.

- **Task Completion Time (TCT)**: End-to-end time from submission to result (seconds).
- **GPU Memory Utilization**: Fraction of GPU memory holding useful KV cache.
- **Throughput**: Completed tasks per minute.
- **SLO Attainment**: Fraction of tasks meeting deadline (1.5× expected time).

Methodology. All experiments repeated 10 times with different random seeds. We report mean ± standard deviation. Statistical significance assessed using two-tailed Welch's t-test; * indicates $p < 0.05$, ** indicates $p < 0.01$, *** indicates $p < 0.001$. Three warm-up runs excluded. Outliers beyond 1.5× IQR removed (<2% of measurements).

9.1.1 Baseline Currency Discussion. Our primary implementation extends vLLM v0.6.0, and we evaluate against the latest releases of all major systems. **Critical comparison:** vLLM v0.15.1 with Automatic Prefix Caching (APC) and PrefixCacheAffinityRouter represents the current state-of-the-art. This configuration addresses prefix sharing and routes requests with similar prefixes to the same workers. As shown in Table 3, vLLM+APC achieves substantial improvements over earlier vLLM versions, but SAGA still achieves 1.72× speedup ($p < 0.001$) because:

(1) **Session vs. prefix affinity:** vLLM's affinity router groups requests by shared prefixes (system prompts, tool definitions)

but does not track session identity. Agent sessions with identical prefixes but different conversation histories are not distinguished. SAGA routes by session ID, ensuring all steps of a task reach the same worker.

(2) **Tool-call TTL:** vLLM's cache uses standard LRU eviction during idle periods. During long tool calls (median 1.2s, P99 45s), the session's KV cache may be evicted under memory pressure. SAGA's workflow-aware TTL predicts tool completion and retains cache accordingly.

(3) **Task-level fairness:** vLLM's scheduling optimizes per-request latency. Under multi-tenant load, light tenants experience starvation. SAGA's AFS scheduling provides completion-time fairness at the task level.

Comparison with TensorRT-LLM Scaffolding: TRT-LLM v1.1's Scaffolding framework addresses multi-step reasoning through KV Cache Connector API. However, Scaffolding focuses on single-node inference-time compute rather than distributed cluster scheduling. SAGA's 1.58× advantage over TRT-LLM+Scaffolding comes from cluster-wide session affinity and work stealing.

Model size discussion. Our evaluation uses Llama-3-70B-Instruct with GQA ($n_{kv} = 8$), yielding ~10.7GB KV cache per 32K session. PRISM's benefits scale with model size because KV cache regeneration cost is proportional to model dimension: for smaller models (8B, ~1.5GB cache), regeneration takes ~0.3s per step, yielding moderate benefits. For larger models (405B, ~50GB+ cache across TP groups), regeneration takes ~5s per step, yielding proportionally larger benefits. MoE architectures require routing-aware extensions (acknowledged in §1.5).

9.2 End-to-End Performance

Table 3 shows end-to-end performance on agent benchmarks with full statistical details.

On SWE-bench, SAGA achieves 3.01×±0.16 speedup over vLLM v0.6.0 ($p < 0.001$). Against the state-of-the-art vLLM+APC baseline (v0.15.1 with Automatic Prefix Caching and affinity routing), SAGA still achieves 1.73× speedup ($p < 0.001$), confirming that workflow-level optimization provides benefits beyond what prefix caching and affinity routing alone can deliver. The 1.47× improvement over KVFlow demonstrates that SAGA's integrated approach (distributed scheduling + TTL policies + AFS fairness) outperforms workflow-aware caching alone.

Breakdown analysis. Figure 1(a) shows the time breakdown for SWE-bench tasks. vLLM v0.6.0 spends 38% of time regenerating KV cache after tool calls. vLLM+APC reduces this to 22% through prefix sharing and affinity routing, but still evicts session-specific cache during long tool calls. SAGA reduces regeneration to 8% through workflow-aware TTL.

9.3 Ablation Study

Table 4 quantifies individual component contributions through ablation experiments on SWE-bench.

Session affinity provides the largest benefit (96% slowdown when disabled), as it directly prevents cache regeneration by routing related requests to the same worker. Workflow-aware eviction and TTL together contribute 42–54% improvement. Speculative prefetching provides 19% improvement by overlapping cache loading with tool execution. AFS contributes a smaller 8% improvement in single-benchmark settings but becomes essential under multi-tenant contention (§9.6).

Table 3: End-to-end performance on agent benchmarks. TCT = Task Completion Time (seconds). Mem = GPU memory utilization (%). Values show mean ± std over 10 trials. Significance: *** $p < 0.001$, ** $p < 0.01$ vs. each baseline (pairwise Welch's t-test).

	SWE-bench		WebArena	
System	**TCT (s)**	**Mem%**	**TCT (s)**	**Mem%**
vLLM v0.6.0	612.3±32.1	42.1±2.3	178.4±14.2	45.3±2.1
vLLM+APC v0.15.1	352.1±21.4	58.7±2.8	127.3±10.1	61.2±2.5
SGLang v0.5.8	387.2±24.3	56.2±2.6	138.7±11.3	58.9±2.4
Llumnix v1.2	498.1±28.7	48.3±2.4	156.2±12.8	51.7±2.2
TRT-LLM+Scaff.	324.6±19.8	61.4±2.7	118.9±9.4	63.8±2.6
vLLM+KVFlow	298.4±18.2	64.1±2.9	108.2±8.7	66.4±2.7
SAGA	**203.4**±12.8	**71.3**±2.4	**82.1**±6.8	**74.6**±2.3
Speedup of SAGA vs. baselines (TCT ratio):				
vs. vLLM v0.6.0	3.01×***		2.17×***	
vs. vLLM+APC	1.73×***		1.55×***	
vs. SGLang	1.90×***		1.69×***	
vs. Llumnix	2.45×***		1.90×***	
vs. TRT-LLM	1.60×***		1.45×**	
vs. KVFlow	1.47×***		1.32×**	
Geometric mean speedup vs. vLLM+APC: **1.64×**				

Table 4: Ablation study on SWE-bench. Each row removes one component from full system. Values show mean ± std over 10 trials.

Configuration	TCT (s)	vs. Full
Full SAGA	203.4±12.8	—
w/o Workflow-aware eviction	312.8±18.3	+54%
w/o Tool-call TTL	289.1±16.7	+42%
w/o Speculative prefetch	241.6±14.2	+19%
w/o Session affinity	398.2±22.4	+96%
w/o Work stealing	267.3±15.9	+31%
w/o AFS fairness	218.7±13.1	+8%

9.4 Pattern Inference Evaluation

Table 5 compares performance with and without framework hints.

Pattern inference achieves 87% accuracy in predicting workflow structure, resulting in 15.6% performance degradation compared to explicit hints—still providing 3.61× speedup over vLLM. The 13% error rate primarily manifests as incorrect successor predictions at branching points (e.g., predicting a "retry" loop when the agent proceeds to a new step), causing unnecessary cache retention for the wrong branch. These errors do not cascade: the system detects misprediction when the actual next request arrives and corrects routing for subsequent steps.

Table 5: Performance comparison: framework hints vs. pattern inference. Accuracy measures the fraction of correctly predicted next-step node transitions in held-out traces.

Mode	TCT (s)	AEG Accuracy	vs. Hints
With hints	203.4±12.8	100%	—
Pattern inference	235.2±14.6	87%	+15.6%
No AEG (baseline)	398.2±22.4	—	+95.8%

Table 6: SLO attainment (% of tasks meeting deadline) by tenant type.

System	Heavy	Medium	Light	Overall
vLLM	89.4	72.1	43.2	67.3
SGLang	91.2	78.6	51.4	73.4
Llumnix	92.8	81.3	58.9	77.2
SAGA	99.1	99.4	98.7	99.2

9.5 Scalability Analysis

SAGA achieves 6.4× speedup scaling from 8 to 64 GPUs (80% efficiency) for fixed workloads, and near-linear weak scaling (0.94× per doubling) up to 512 concurrent agents.

9.6 Multi-Tenant Fairness

We evaluate multi-tenant behavior using the BurstGPT-derived workload with 10 tenants of varying intensity.

SLO attainment. Table 6 shows SLO attainment (tasks completing within 1.5× expected time). SAGA achieves 99.2% overall attainment, compared to 67.3% for vLLM. The improvement is most dramatic for light tenants (98.7% vs. 43.2%), validating Theorem 2.

Fairness analysis. vLLM exhibits high variance with a long tail for light tenants (P99 = 12.4× expected TCT). SAGA provides consistent completion times across all tenant types (P99 < 1.8× expected).

9.7 Work Stealing Effectiveness

We compare load balance with and without work stealing on a 32-GPU subset (reduced from 64 GPUs to isolate execution model effects from scaling effects). Without work stealing, worker utilization ranges from 23% to 94%. With work stealing, utilization converges to 68–79%.

Migration overhead is minimal: mean 230ms (P95: 890ms), occurring 2.3 times per task on average.

9.8 System Overhead Analysis

Table 7 breaks down SAGA's scheduling overhead.

Total coordinator CPU overhead is 4.2%, leaving 95.8% for application workloads. AFS computation scales linearly with tenant count but remains negligible (3.1ms for 32 tenants).

Table 7: SAGA overhead breakdown (64 GPUs, 32 tenants).

Component	Mean (ms)	P95 (ms)	CPU (%)
Coordinator cycle	12.3	28.7	4.2
AFS computation	3.1	8.4	—
AEG construction	45.2	112.8	—
Work stealing (migration)	230	890	—

Table 8: Execution strategy comparison on SWE-bench (32 GPUs).

Strategy	TCT (s)	Throughput	Evict Rate
Pure BFS	487.2±28.4	12.4 t/m	78%
Pure DFS	623.1±34.2	4.2 t/m	3%
Hybrid (SAGA)	203.4±12.8	8.7 t/m	12%

Table 9: Parameter sensitivity analysis on SWE-bench. TCT_Δ shows maximum variation within the tested range relative to default.

Parameter	Default	Range	TCT_Δ	Justification
α (recency wt.)	0.3	[0.2, 0.4]	<5%	Sensitivity analysis
β (reuse wt.)	0.5	[0.4, 0.6]	<8%	Sensitivity analysis
γ (size wt.)	0.2	[0.1, 0.3]	<3%	Size as tiebreaker
θ (routing)	0.8	[0.6, 0.95]	<5%	20% headroom [58]
$threshold_{low}$	0.7	[0.6, 0.8]	<4%	Soft pressure onset [31]
$threshold_{high}$	0.9	[0.85, 0.95]	<6%	Hard eviction limit
T_{idle} (steal trigger)	100ms	[50, 200]ms	<7%	Amortize steal cost
R_{max} (load ratio)	2.0	[1.5, 3.0]	<4%	Imbalance tolerance
TTL_{max}	300s	[120, 600]s	<3%	P99 tool latency cap
θ_{conf} (AEG)	0.7	[0.5, 0.9]	<6%	Precision-recall tradeoff

9.9 Execution Strategy Tradeoffs

Table 8 compares different execution strategies on SWE-bench with 32 GPUs (reduced scale to isolate strategy effects from cluster-scale effects). The BFS-DFS tradeoff—a well-studied phenomenon in parallel systems [21]—manifests strongly in agent scheduling.

Pure BFS maximizes throughput (12.4 tasks/min) but suffers 78% eviction rates. SAGA's hybrid approach achieves optimal TCT (203.4s) at 30% lower throughput—appropriate for latency-sensitive interactive deployments [20, 72].

9.10 Parameter Sensitivity

Table 9 summarizes sensitivity analysis for all configurable parameters.

All parameters show less than 8% TCT variation within reasonable ranges, indicating that SAGA is robust to parameter choice. This robustness arises because the dominant benefit (session affinity, contributing 96% of improvement per ablation) is largely parameter-free. The most sensitive parameters are β (reuse weight) and T_{idle} (steal trigger), both of which directly affect cache retention and load balance.

Table 10: Sensitivity to tool latency variance (coefficient of variation). Mean tool latency held constant at 1.2s.

CV	TCT (s)	TTL Accuracy	Evict Rate	vs. CV=0.5
0.5	195.1±11.2	96%	9%	—
1.0	203.4±12.8	93%	12%	+4%
1.5	218.6±15.3	88%	18%	+12%
2.0	241.3±18.7	82%	24%	+24%
3.0	298.4±24.1	71%	35%	+53%

Table 11: Comparison with directly related program-aware LLM serving systems.

System	Venue	Distributed Scheduling	Tool TTL Policies	Fairness Guarantee	Competitive Ratio
Parrot [35]	OSDI'24	×	×	×	×
Autellix [37]	arXiv'25	✓	×	×	×
Pie [64]	SOSP'25	×	✓	×	×
KVFlow [46]	arXiv'25	×	✓	×	×
SAGA	**HPDC'26**	✓	✓	✓	✓

9.11 Tool Latency Variance Sensitivity

Table 10 shows how SAGA's performance varies with tool latency variance, measured by coefficient of variation (CV) of tool call durations. We synthetically vary CV while keeping mean tool latency constant.

SAGA's adaptive TTL maintains consistent performance up to CV=2.0 (24% TCT degradation). Beyond CV=2.0, TTL prediction accuracy degrades significantly as extreme outliers cause premature eviction. In practice, production tool latencies have CV≈1.0–1.5 (Table 1), well within SAGA's effective range.

10 Related Work

Table 11 compares SAGA with directly related program-aware systems.

LLM serving. vLLM [31], SGLang [70], Orca [68], and TensorRT-LLM [44] optimize request-level metrics; SAGA adds workflow-level optimization.

Program-aware serving. Parrot [35] introduces Semantic Variables but lacks distributed scheduling and fairness. Autellix [37] proposes program-level fairness; Pie [64] decomposes generation via inferlets. SAGA provides the first integrated framework combining distributed scheduling, tool-aware TTL, fairness guarantees, and competitive ratio analysis.

Distributed inference. Llumnix [58] enables KV migration; SOLA [23] optimizes SLO attainment. KVFlow [46] and Continuum [27] introduce workflow-aware eviction. SAGA extends these with cluster-wide coordination and formal guarantees.

Fairness and caching. DRF [18], VTC [54], and Themis [38] address resource fairness; SAGA extends to task-completion fairness. Our WA-LRU achieves 1.31× competitive ratio against Bélády's optimal [4].

11 Conclusions and Future Work

We presented SAGA, a distributed scheduling system for multi-step AI agent workloads that treats agent programs as first-class schedulable units. By adapting three classical systems principles—workflow scheduling, informed caching, and application-level fairness—to the compound AI domain with domain-specific technical innovations, SAGA achieves $1.73 \times \pm 0.11$ and $1.55 \times \pm 0.09$ task completion time reductions on SWE-bench and WebArena respectively compared to vLLM v0.15.1 with Automatic Prefix Caching (geometric mean: 1.64×, $p < 0.001$), and $1.22 \times \pm 0.05$ memory utilization improvement, while providing 99.2% SLO attainment under multi-tenant interference. Against systems without workflow awareness, improvements reach 3.01×. These gains come at the cost of approximately 25% throughput reduction, a tradeoff appropriate for the latency-sensitive interactive deployments that dominate compound AI usage.

Technical contributions. We formalized Agent Execution Graphs for capturing workflow structure, and showed that WA-LRU eviction achieves within 1.31× of Bélády's optimal offline policy—the first empirical competitive ratio analysis for workflow-aware KV cache management. Our Lyapunov-drift-based analysis of AFS provides formal completion time bounds with explicit derivation of the restoring drift property.

Positioning relative to concurrent work. SAGA complements recent systems addressing program-aware LLM serving (Parrot, Autellix, Pie) by providing the first integrated framework combining distributed cluster-wide scheduling, tool-call-aware TTL policies, formal fairness guarantees, and competitive ratio analysis. Table 11 details these distinctions.

Open research directions. This work opens several directions:

(1) **Optimal TTL prediction:** Can we learn TTL policies with provable regret bounds? The problem resembles online learning with partial feedback [33].
(2) **Tighter competitive ratios:** Our empirical 1.31× competitive ratio against Bélády suggests room for improvement. What is the information-theoretic lower bound achievable with AEG predictions?
(3) **Complexity of workflow-aware scheduling:** Is optimal workflow-aware scheduling with cache constraints NP-hard? A formal complexity result would justify heuristic approaches and guide algorithm design.
(4) **Geo-distributed scheduling:** How should AFS extend when agents span datacenters with network-dependent cache migration costs?
(5) **Multi-agent coordination:** When agents interact (collaborative coding, negotiation), how should their execution graphs be jointly optimized?
(6) **Speculation integration:** Combining speculative execution [51, 67] with workflow-aware scheduling could provide multiplicative benefits.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, Article 7 (2024), 18 pages.

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023* (2023), 4895–4901. doi:10.18653/V1/2023.EMNLP-MAIN.298

[3] Amazon Web Services. 2024. Amazon Q Developer. AWS Product Page. (2024). https://aws.amazon.com/q/developer/

[4] L. A. Belady. 2010. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal* 5 (2010). Issue 2. doi:10.1147/sj.52.0078

[5] Robert D. Blumofe and Charles E. Leiserson. 1999. Scheduling Multithreaded Computations by Work Stealing. *J. ACM* 46, 5 (1999), 720–748. doi:10.1145/324133.324234

[6] Pei Cao, Edward W. Felten, Anna R. Karlin, and Kai Li. 1996. Implementation and Performance of Integrated Application-Controlled File Caching, Prefetching, and Disk Scheduling. *ACM Trans. Comput. Syst.* 14, 4 (1996), 311–343. doi:10.1145/235543.235544

[7] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson C. Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. 2013. Spanner: Google's Globally Distributed Database. *ACM Trans. Comput. Syst.* 31, 3 (2013), 8. doi:10.1145/2491245

[8] Christian Corrò and Luca Chittaro. 2025. Exploring the Potential and Limitations of Large Language Models to Control the Behavior of Embodied Persuasive Agents. *Persuasive Technology: 20th International Conference, PERSUASIVE 2025, Limassol, Cyprus, May 5-7, 2025, Proceedings* (2025), 61–73. doi:10.1007/978-3-031-94959-3_5

[9] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *12th International Conference on Learning Representations, ICLR 2024* (2024).

[10] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022* (2022).

[11] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and R. Kent Wenger. 2015. Pegasus, a workflow management system for science automation. *Future Gener. Comput. Syst.* 46 (2015), 17–35. doi:10.1016/J.FUTURE.2014.10.008

[12] Peter J. Denning. 2014. Virtual Memory. *Computing Handbook, Third Edition: Computer Science and Software Engineering* (2014), 54: 1–21.

[13] U Drepper. 2007. What every programmer should know about memory. *Red Hat* (2007). doi:10.1.1.91.957

[14] Devdatt P. Dubhashi and Alessandro Panconesi. 2009. *Concentration of Measure for the Analysis of Randomized Algorithms.* Cambridge University Press.

[15] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23 (2022).

[16] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024* (2024), 135–153.

[17] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024).

[18] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. 2011. Dominant Resource Fairness: Fair Allocation of Multiple Resource Types. *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2011, Boston, MA, USA, March 30 - April 1, 2011* (2011).

[19] In Gim, Guojun Chen, Seung-Seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024* (2024).

[20] GitHub Next. 2024. Copilot Workspace: A Copilot-Native Dev Environment. (April 2024). https://github.blog/news-insights/product-news/github-copilot-workspace/

[21] Goetz Graefe. 1990. Encapsulation of parallelism in the Volcano query processing system. (1990). doi:10.1145/93597.98720

[22] Maurice Herlihy. 2006. *The art of multiprocessor programming.* doi:10.1145/1146381.1146382

[23] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Guohao Dai, Xuefei Ning, Shengen Yan, Yun Liang, and Yu Wang. 2025. SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling. *Proceedings of the Eighth Conference on Machine Learning and Systems, MLSys 2025, Santa Clara, CA, USA, May 12-15, 2025* (2025).

[24] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads. *arXiv preprint* (2024). arXiv:2401.11181

[25] InfiniBand Trade Association. 2020. InfiniBand Architecture Specification, Volume 2, Release 1.4. (2020). Physical and electrical specifications. Available to IBTA members.

[26] Patrick Jaillet, Jiashuo Jiang, Konstantina Mellou, Marco Molinaro, Chara Podimata, and Zijie Zhou. 2025. Online Scheduling for LLM Inference with KV Cache Constraints. *arXiv preprint* (2025). arXiv:2502.07115

[27] Matthijs Jansen, Linus Wagner, Animesh Trivedi, and Alexandru Iosup. 2023. Continuum: Automate Infrastructure Deployment and Benchmarking in the Compute Continuum. *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering, ICPE 2023, Coimbra, Portugal, April 15-19, 2023* (2023), 181–188. doi:10.1145/3578245.3584936

[28] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues? *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024* (2024).

[29] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2026. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. *ACM Transactions on Computer Systems* 44 (2026). Issue 1. doi:10.1145/3768628

[30] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2025. AI Agents That Matter. *Trans. Mach. Learn. Res.* 2025 (2025).

[31] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023* (2023), 611–626. doi:10.1145/3600006.3613165

[32] Inc. LangChain. 2022. LangChain: Build Context-Aware Reasoning Applications. (2022). https://github.com/langchain-ai/langchain

[33] Tor Lattimore and Csaba Szepesvári. 2020. *Bandit algorithms.* Cambridge University Press.

[34] Yueying Li, Jian Gang Dai, and Tianyi Peng. 2025. Throughput-Optimal Scheduling Algorithms for LLM Inference and AI Agents. *arXiv preprint* (2025). arXiv:2504.07347

[35] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024* (2024), 929–945.

[36] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. *Proceedings of the ACM SIGCOMM 2024 Conference, ACM SIGCOMM 2024, Sydney, NSW, Australia, August 4-8, 2024* (2024), 38–56. doi:10.1145/3651890.3672274

[37] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2025. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. *arXiv preprint* (2025). arXiv:2502.13965

[38] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. 2020. THEMIS: Fair and efficient GPU cluster scheduling. *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020* (2020).

[39] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. *Proceedings of the FAST '03 Conference on File and Storage Technologies, March 31 - April 2, 2003, Cathedral Hill Hotel, San Francisco, California, USA* (2003).

[40] Meta AI. 2024. Llama 3 Model Card. *Meta Llama Documentation* (2024). https://www.llama.com/docs/model-cards-and-prompt-formats/meta-llama-3/

[41] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018* (2018), 561–577.

[42] João Moura and CrewAI Inc. 2023. CrewAI: Framework for Orchestrating Role-Playing, Autonomous AI Agents. (2023). Open-sourced November 2023. Accessed: 2026-02-06.

[43] NVIDIA Corporation. 2018. NVIDIA NVSwitch: The World's Highest-Bandwidth On-Node Switch. *NVIDIA* (2018). https://images.nvidia.com/content/pdf/nvswitch-technical-overview.pdf

[44] NVIDIA Corporation. 2023. TensorRT-LLM: High-Performance Large Language Model Inference. (2023). https://github.com/NVIDIA/TensorRT-LLM

[45] Kay Ousterhout, Patrick Wendell, Matei Zaharia, and Ion Stoica. 2013. Sparrow: distributed, low latency scheduling. *ACM SIGOPS 24th Symposium on Operating Systems Principles, SOSP '13, Farmington, PA, USA, November 3-6, 2013* (2013), 69–84. doi:10.1145/2517349.2522716

[46] Zaifeng Pan, AJJKUMAR PATEL, Yipeng Shen, Zhengding Hu, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. *The Thirty-ninth Annual Conference on Neural Information Processing Systems* (2025).

[47] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024* (2024), 118–132. doi:10.1109/ISCA59077.2024.00019

[48] R. Hugo Patterson, Garth A. Gibsonf, Eka Gintingf, Daniel Stodolskyt, and Jim Zelenkat. 1995. Informed prefetching and caching. *Proceedings of the 15th ACM Symposium on Operating Systems Principles, SOSP 1995* (1995). doi:10.1145/224056.224065

[49] Steve Rennich. 2012. CUDA C/C++ Streams and Concurrency. *NVIDIA CUDA Training Webinar* (2012). https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf

[50] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. 2016. VDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. *Proceedings of the Annual International Symposium on Microarchitecture, MICRO* 2016-December (2016). doi:10.1109/MICRO.2016.7783721

[51] Yeonju Ro, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, Aditya Akella, Zhangyang Wang, Mattan Erez, and Esha Choukse. 2025. Sherlock: Reliable and Efficient Agentic Workflow Execution. *arXiv preprint* (2025). arXiv:2511.00330

[52] Matthew Rocklin. 2015. Dask: Parallel Computation with Blocked algorithms and Task Scheduling. *Proceedings of the 14th Python in Science Conference, SciPy 2015, Austin, Texas, USA, July 6-12, 2015* (2015), 126–132. doi:10.25080/MAJORA-7B98E3ED-013

[53] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024* (2024).

[54] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024* (2024), 965–988.

[55] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA* 202 (2023), 31094–31116.

[56] Daniel Dominic Sleator and Robert Endre Tarjan. 1985. Amortized Efficiency of List Update and Paging Rules. *Commun. ACM* 28, 2 (1985), 202–208. doi:10.1145/2786.2793

[57] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. *IEEE International Symposium on High Performance Computer Architecture, HPCA 2025, Las Vegas, NV, USA, March 1-5, 2025* (2025), 1348–1362. doi:10.1109/HPCA61900.2025.00102

[58] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024* (2024), 173–191.

[59] Douglas Thain, Todd Tannenbaum, and Miron Livny. 2005. Distributed computing in practice: the Condor experience. *Concurr. Pract. Exp.* 17, 2-4 (2005), 323–356. doi:10.1002/CPE.938

[60] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 2017-December (2017). doi:10.1201/9781003561460-19

[61] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* 2 (2025). doi:10.1145/3711896.3737413

[62] Fangzhou Wu, Sandeep Silwal, Qiuyi, and Zhang. 2026. Randomization Boosts KV Caching, Learning Balances Query Load: A Joint Perspective. *arXiv preprint* (2026). arXiv:2601.18999 [cs.LG]

[63] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. *arXiv preprint* (2023). arXiv:2308.08155

[64] Yi Xu, Ziming Mao, Xiangxi Mo, Shu Liu, and Ion Stoica. 2024. Pie: Pooling CPU Memory for LLM Inference. *arXiv preprint* (2024). arXiv:2411.09317

[65] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023).

[66] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023* (2023).

[67] Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. 2025. Speculative Actions: A Lossless Framework for Faster Agentic Systems. *arXiv preprint* (2025). arXiv:2510.04371

[68] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022* (2022), 521–538.

[69] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. 2024. The Shift from Models to Compound AI Systems. *https://bair.berkeley.edu* (2024).

[70] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024).

[71] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024* (2024), 193–210.

[72] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024* (2024).